



Documentación Completa de Tests — SymptoLeaf

Fecha de creación: 2026-02-11

Proyecto: SymptoLeaf — Detección de enfermedades en plantas mediante IA

Versión del código base: `main.dart` actualizado con AuthViewModel + DiagnosticsViewModel

Total de archivos de test: 19 (adaptados a versión actual)

Total de tests individuales: 190

Estado:  Todos los tests pasan (`flutter test` → 190 passed)



Índice

1. [Introducción y Propósito](#)
2. [Estructura de los Tests](#)
3. [Compatibilidad con el Código Actual \(`main.dart` \)](#)
4. [Tests Unitarios](#)
 - 4.1 [PredictionEntity](#)
 - 4.2 [PredictionModel](#)
 - 4.3 [ChatMessageModel](#)
 - 4.4 [TreatmentModel](#)
 - 4.5 [FotoProvider](#)
 - 4.6 [PredictDiseaseUseCase](#)
 - 4.7 [PredictionViewModel](#)
 - 4.8 [GeminiViewModel](#)
 - 4.9 [SettingsViewModel](#)
5. [Tests de Integración](#)
 - 5.1 [Flujo de Análisis E2E](#)
 - 5.2 [Flujo de Datos](#)
 - 5.3 [Flujo de Gemini \(Chat/Tratamientos\)](#)
 - 5.4 [Flujo de Predicción](#)
 - 5.5 [Providers y Estado Global](#)
6. [Tests de Rendimiento \(Performance\)](#)
 - 6.1 [Benchmark de Inferencia](#)
 - 6.2 [Carga Extrema \(Stress\)](#)
 - 6.3 [Memoria y Sesiones Prolongadas](#)
7. [Tests de Regresión](#)
 - 7.1 [Modelo ONNX y Clases](#)
8. [Tests de Widget](#)
 - 8.1 [HomeScreen](#)
9. [Cómo Ejecutar los Tests](#)
10. [Resumen de Cobertura](#)

1. Introducción y Propósito

¿Qué es un test (prueba)?

Un **test** es un fragmento de código que **verifica automáticamente** que otra parte del código funciona correctamente. En lugar de probar la app manualmente (abrir la app, tocar botones, ver si funciona), los tests lo hacen solos y te dicen si algo falla.

¿Por qué se hacen tests?

Razón	Explicación sencilla
Detectar errores temprano	Si cambias algo y un test falla, sabes que rompiste algo antes de que el usuario lo vea
Documentar comportamiento	Los tests dicen exactamente qué debe hacer cada parte del código
Confianza al modificar	Puedes cambiar código sin miedo: si los tests pasan, todo sigue funcionando
Calidad del software	Un proyecto con tests bien hechos es más profesional y confiable

¿Cómo funciona un test?

Cada test sigue el patrón **AAA (Arrange-Act-Assert)**:

```
test('descripción de lo que se prueba', () {
  // Arrange (Preparar) – Se crean los objetos necesarios
  final objeto = MiObjeto();

  // Act (Actuar) – Se ejecuta la acción que queremos probar
  final resultado = objeto.hacerAlgo();

  // Assert (Verificar) – Se comprueba que el resultado es el esperado
  expect(resultado, equals(valorEsperado));
});
```

Estructura de este proyecto de tests

Los tests están organizados en **5 categorías principales**, cada una probando un aspecto diferente de la aplicación:

```
test/
├─ unit/           → Pruebas de piezas individuales (modelos, entidades, viewmodels)
├─ integration/    → Pruebas de múltiples componentes trabajando juntos
├─ performance/    → Pruebas de velocidad, carga y memoria
├─ regression/     → Pruebas para asegurar que el modelo de IA no cambie
└─ widget/         → Pruebas de la interfaz visual (pantallas)
```

2. Estructura de los Tests

Mapa completo de archivos

#	Categoría	Archivo	Tests	Descripción
1	Unit	unit/entities/prediction_entity_test.dart	10	Entidad de predicción (dominio)
2	Unit	unit/models/prediction_model_test.dart	10	Modelo de predicción (datos)
3	Unit	unit/models/chat_message_model_test.dart	10	Modelo de mensajes de chat
4	Unit	unit/models/treatment_model_test.dart	10	Modelo de tratamientos
5	Unit	unit/providers/foto_provider_test.dart	10	Provider de fotografías
6	Unit	unit/use_cases/predict_disease_usecase_test.dart	10	Caso de uso de predicción
7	Unit	unit/viewmodels/prediction_viewmodel_test.dart	10	ViewModel de predicciones
8	Unit	unit/viewmodels/gemini_viewmodel_test.dart	10	ViewModel de Gemini
9	Unit	unit/viewmodels/settings_viewmodel_test.dart	10	ViewModel de configuración (ModelType)
10	Integration	integration/analysis_flow_test.dart	8	Flujo E2E de análisis
11	Integration	integration/data_flow_test.dart	10	Flujo de datos completo
12	Integration	integration/gemini_flow_test.dart	10	Flujo de Gemini chat
13	Integration	integration/prediction_flow_test.dart	10	Flujo de predicción
14	Integration	integration/providers_state_test.dart	10	Estado global de providers
15	Performance	performance/inference_benchmark_test.dart	10	Métricas de rendimiento
16	Performance	performance/load_stress_test.dart	12	Carga extrema 100+ operaciones
17	Performance	performance/memory_stress_test.dart	10	Memoria y leaks
18	Regression	regression/onnx_model_test.dart	10	Regresión del modelo ONNX
19	Widget	widget/home_screen_test.dart	10	Pantalla principal

3. Compatibilidad con el Código Actual (main.dart)

Los tests fueron originalmente escritos para una versión anterior del proyecto (testing_Symptoleaf). Se adaptaron para ser compatibles con la versión actual de symptoLeaf . A continuación se detallan los cambios realizados:

Cambios aplicados durante la adaptación

Aspecto	Versión Testing (antigua)	Versión Actual (main.dart)	Adaptación realizada
Enum de configuración	PredictionMode (local/server)	ModelType (standard/yolo11)	Reemplazado en todos los tests
Getter de modo	settings.mode	settings.modelType	Reemplazado en todos los tests
Setter de modo	settings.setMode() (sync)	settings.setModelType() (async)	Reemplazado + añadido await
URL del servidor	settings.serverUrl / setServerUrl()	Eliminado (ya no existe)	Tests adaptados a modelName
WelcomeScreen	Existía	No existe (reemplazada por LoginScreen)	Test eliminado
AuthViewModel	No existía	Existe (auth_viewmodel.dart)	No requiere cambio (no probado)
DiagnosticsViewModel	No existía	Existe (diagnostics_viewmodel.dart)	No requiere cambio (no probado)
OnnxDataSource constructor	OnnxDataSource() sin params	OnnxDataSource(modelFileName: ...)	No afecta tests (no usan OnnxDataSource directo)

Archivos eliminados (incompatibles)

Archivo	Razón
test/widget/welcome_screen_test.dart	WelcomeScreen ya no existe en el proyecto
test/gemini_test.dart	Archivo vacío/sin contenido
test/widget_test.dart	Test por defecto de Flutter (counter app), no aplica

Estado actual:  TODOS los 190 tests pasan

```
flutter test
# 00:19 +190: All tests passed!
```

4. Tests Unitarios

Los tests unitarios prueban **una sola pieza del código a la vez**, aislada del resto. Son los más básicos y fundamentales.

4.1 prediction_entity_test.dart

Archivo: test/unit/entities/prediction_entity_test.dart

Qué se prueba: La entidad PredictionEntity del dominio (capa de negocio)

Ubicación del código probado: lib/domain/entities/prediction_entity.dart

¿Por qué se hace esta prueba?

PredictionEntity es el **objeto principal** que representa el resultado de analizar una planta. Es la entidad del dominio — la representación pura del dato sin lógica de framework ni base de datos. Si esta entidad no funciona correctamente, **todo el sistema de predicciones falla**.

¿Qué nos asegura?

Que podemos crear objetos de predicción con todos sus campos, que los valores se almacenan correctamente, y que las listas de top 3 predicciones funcionan como se espera.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-PE-001	Crear entidad con todos los campos	Crea una PredictionEntity con className= Tomato_Late_blight , plant= Tomato , disease= Late blight , confidence= 0.95 , isHealthy= false	Que cada campo se guarde correctamente (className , plant , disease , confidence , isHealthy)	Si un campo no se guarda bien, el resultado que ve el usuario será incorrecto
UT-PE-002	Crear entidad de planta saludable	Crea una entidad donde isHealthy=true	Que la propiedad isHealthy sea true	La app muestra un mensaje diferente si la planta está sana. Si este valor no funciona, diría que una planta sana está enferma
UT-PE-003	Crear entidad con top3 predicciones	Crea una entidad con una lista de 3 predicciones alternativas	Que top3.length == 3 y que están ordenadas por confianza descendente	El usuario ve las 3 posibles enfermedades; si la lista no se llena bien, el diagnóstico alternativo no funciona
UT-PE-004	Confianza entre 0 y 1	Crea entidades con confidence 0.0 y 1.0	Que los valores extremos se acepten sin error	Si no acepta 0.0 o 1.0, el sistema fallaría en predicciones muy seguras o muy inseguras

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-PE-005	Planta requerida	Crea una entidad y verifica que <code>plant</code> no esté vacío	Que <code>entity.plant.isNotEmpty</code>	Sin nombre de planta, el usuario no sabría qué planta analizó
UT-PE-006	Crear <code>PredictionTop3</code>	Crea un objeto <code>PredictionTop3</code> individual	Que todos los campos (<code>className</code> , <code>plant</code> , <code>disease</code> , <code>confidence</code> , <code>isHealthy</code>) se guarden	Cada elemento del top 3 debe tener datos completos
UT-PE-007	<code>PredictionTop3</code> saludable	Crea un <code>PredictionTop3</code> con <code>isHealthy=true</code>	Que <code>isHealthy</code> funcione en el sub-objeto	Las alternativas también pueden ser "saludable"
UT-PE-008	Diferentes plantas	Crea entidades para Tomato, Potato, Apple, Grape, Corn	Que cada planta se almacene correctamente	El modelo soporta 5 plantas; todas deben funcionar
UT-PE-009	Confianza alta vs baja	Crea entidades con <code>confidence</code> <code>0.95</code> y <code>0.35</code>	Que la alta sea <code>> 0.5</code> y la baja <code>< 0.5</code>	La app podría mostrar advertencias según la confianza
UT-PE-010	Lista top3 vacía válida	Crea entidad con <code>top3: []</code> (lista vacía)	Que <code>top3.isEmpty</code> sea <code>true</code>	En modo local a veces no hay alternativas; la lista vacía debe ser válida

4.2 prediction_model_test.dart

Archivo: `test/unit/models/prediction_model_test.dart`

Qué se prueba: El modelo `PredictionModel` de la capa de datos

Ubicación del código probado: `lib/data/models/prediction_model.dart`

¿Por qué se hace esta prueba?

`PredictionModel` es el objeto que **recibe los datos del servidor o del modelo ONNX** y los estructura para ser usados en la app. Tiene la lógica de serialización (convertir JSON ↔ objeto) y conversión a entidad. Si falla, la app no puede interpretar los resultados del modelo de IA.

¿Qué nos asegura?

Que los datos que llegan del modelo de predicción se parsean correctamente, se pueden serializar a JSON para almacenar, y se convierten sin pérdida a la entidad del dominio.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-PM-001	Crear modelo con datos válidos	Crea <code>PredictionModel</code> con <code>className</code> , <code>plant</code> , <code>disease</code> , <code>confidence</code> , <code>isHealthy</code> , <code>top3</code>	Que todos los campos se almacenen exactamente como se proporcionaron	Base del modelo de datos; sin esto nada funciona
UT-PM-002	Crear modelo con top3	Crea modelo con 3 items en la lista <code>top3</code>	Que <code>top3.length == 3</code> y que están en orden descendente de confianza	El usuario ve las alternativas; el orden importa
UT-PM-003	Modelo saludable	Crea modelo con <code>isHealthy=true</code> y <code>disease='Saludable'</code>	Que <code>isHealthy</code> sea <code>true</code>	Distinguir planta sana de enferma
UT-PM-004	Confianza en límite inferior	Crea modelo con <code>confidence=0.0</code>	Que acepte <code>0.0</code> y sea <code>>= 0.0</code>	Un modelo puede no detectar nada (0% confianza)
UT-PM-005	Confianza en límite superior	Crea modelo con <code>confidence=1.0</code>	Que acepte <code>1.0</code> y sea <code><= 1.0</code>	Predicción 100% segura
UT-PM-006	Confianza en rango válido	Prueba múltiples valores: 0.0, 0.25, 0.5, 0.75, 0.95, 1.0	Que cada valor esté entre 0.0 y 1.0	Cobertura completa del rango de confianza
UT-PM-007	Conversión a entidad (<code>toEntity</code>)	Crea modelo y llama <code>model.toEntity()</code>	Que la entidad resultante tenga los mismos valores que el modelo	Esta conversión conecta la capa de datos con la de dominio; si pierde datos, el resultado es incorrecto
UT-PM-008	Serialización a JSON (<code>toJson</code>)	Crea modelo y llama <code>model.toJson()</code>	Que el JSON tenga la estructura correcta con <code>prediction.class</code> , <code>prediction.plant</code> , etc.	Necesario para almacenar o enviar resultados
UT-PM-009	Top3 model básico	Crea <code>PredictionTop3Model</code> individualmente	Que todos sus campos funcionen	Cada alternativa en top3 es un modelo independiente
UT-PM-010	Top3 conversión a entidad	Crea <code>PredictionTop3Model</code> y llama <code>toEntity()</code>	Que preserve <code>className</code> y <code>confidence</code> al convertir	La conversión top3 model → entity debe ser sin pérdida

4.3 chat_message_model_test.dart

Archivo: `test/unit/models/chat_message_model_test.dart`

Qué se prueba: El modelo `ChatMessage` y el enum `MessageRole`

Ubicación del código probado: lib/data/models/chat_message_model.dart

¿Por qué se hace esta prueba?

El chat con Gemini es una funcionalidad central de la app donde los usuarios hacen preguntas sobre tratamientos. `ChatMessage` representa cada mensaje en la conversación. Si los mensajes no se crean correctamente, el chat no mostraría las respuestas del asistente o confundiría quién dijo qué.

¿Qué nos asegura?

Que los mensajes se crean correctamente con su contenido, rol (usuario o asistente), timestamp, y que las factory methods (`user()`, `assistant()`, `loading()`) funcionan adecuadamente.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-CM-001	Crear mensaje de usuario	Crea <code>ChatMessage</code> con <code>role= user</code>	Que <code>isUser=true</code> , <code>isAssistant=false</code> , <code>isLoading=false</code> , contenido correcto	El chat debe identificar quién escribió cada mensaje
UT-CM-002	Crear mensaje de asistente	Crea <code>ChatMessage</code> con <code>role= assistant</code>	Que <code>isAssistant=true</code> , <code>isUser=false</code>	Distinguir respuestas de Gemini de las preguntas del usuario
UT-CM-003	Timestamp automático	Crea mensaje y verifica que el timestamp esté entre <code>beforeCreation</code> y <code>afterCreation</code>	Que se asigne timestamp automáticamente dentro del rango correcto	Los mensajes deben tener hora para ordenarse cronológicamente
UT-CM-004	Timestamp personalizado	Crea mensaje con timestamp <code>DateTime(2026, 1, 15, 10, 30)</code>	Que el timestamp sea exactamente el proporcionado	Permite reconstruir historial de chat desde almacenamiento
UT-CM-005	Factory <code>ChatMessage.user()</code>	Usa el factory method <code>ChatMessage.user('texto')</code>	Que cree un mensaje de usuario con role correcto	Atajo para crear mensajes; debe funcionar igual que el constructor
UT-CM-006	Factory <code>ChatMessage.assistant()</code>	Usa <code>ChatMessage.assistant('texto')</code>	Que cree un mensaje de asistente correctamente	Atajo para respuestas de Gemini

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-CM-007	Factory ChatMessage.loading()	Usa ChatMessage.loading()	Que tenga contenido vacío, role=assistant, isLoading=true	Mientras Gemini responde, se muestra un indicador de carga
UT-CM-008	Getter isUser	Crea mensaje de usuario y de asistente	Que isUser sea true solo para usuario	Evitar que ambos roles devuelvan true
UT-CM-009	Getter isAssistant	Crea mensaje de usuario y de asistente	Que isAssistant sea true solo para asistente	Complemento del test anterior; seguridad doble
UT-CM-010	Verificar roles disponibles	Consulta MessageRole.values	Que haya exactamente 2 roles: user y assistant	Si alguien agrega un rol nuevo sin actualizar el chat, esto lo detecta

4.4 treatment_model_test.dart

Archivo: test/unit/models/treatment_model_test.dart

Qué se prueba: TreatmentModel , TreatmentOption , TreatmentType y el parseo de respuestas de Gemini

Ubicación del código probado: lib/data/models/treatment_model.dart

¿Por qué se hace esta prueba?

Cuando se detecta una enfermedad, Gemini sugiere tratamientos. TreatmentModel estructura esa información (síntomas, tratamientos orgánicos/químicos/culturales, prevención). Si esta estructura falla, el usuario no recibiría recomendaciones de tratamiento o las recibiría mal formateadas.

¿Qué nos asegura?

Que los tratamientos se estructuran correctamente, que los 3 tipos de tratamiento existen, que las etiquetas con emojis se generan bien, y que las respuestas de Gemini se pueden parsear.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-TM-001	Crear modelo con campos requeridos	Crea TreatmentModel con enfermedad, planta, síntomas, tratamientos y prevención	Que todos los campos se almacenen correctamente	Estructura base de los tratamientos

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-TM-002	Modelo con síntomas vacíos	Crea modelo con listas vacías	Que funcione sin errores con listas vacías	Plantas saludables no tienen síntomas; la app no debe romperse
UT-TM-003	Modelo con tratamientos orgánicos	Crea modelo con 2 tratamientos de tipo <code>TreatmentType.organic</code>	Que los tratamientos orgánicos se almacenen correctamente	Muchos usuarios prefieren tratamientos orgánicos
UT-TM-004	Modelo con tratamientos químicos	Crea modelo con tratamiento <code>TreatmentType.chemical</code>	Que el tipo sea <code>chemical</code>	Los tratamientos químicos son una opción válida
UT-TM-005	Verificar tipos de tratamiento	Consulta <code>TreatmentType.values</code>	Que existan exactamente 3 tipos: <code>organic</code> , <code>chemical</code> , <code>cultural</code>	Si falta un tipo, tratamientos no se pueden clasificar
UT-TM-006	Crear opción de tratamiento	Crea <code>TreatmentOption</code> tipo <code>cultural</code> con nombre y descripción	Que todos los campos funcionen	Cada opción de tratamiento debe estar completa
UT-TM-007	<code>typeLabel</code> para orgánico	Verifica <code>organic.typeLabel</code>	Que contenga 🌱 y Orgánico	La UI muestra emojis para identificar tipo; deben ser correctos
UT-TM-008	<code>typeLabel</code> para químico	Verifica <code>chemical.typeLabel</code>	Que contenga 💊 y Químico	Identificación visual en la interfaz
UT-TM-009	<code>typeLabel</code> para cultural	Verifica <code>cultural.typeLabel</code>	Que contenga 🌿 y Cultural	Identificación visual en la interfaz
UT-TM-010	Parsear respuesta de Gemini	Llama <code>TreatmentModel.fromGeminiResponse()</code> con texto de ejemplo	Que el <code>plantName</code> , <code>diseaseName</code> y <code>additionalInfo</code> se extraigan correctamente	Conecta la respuesta raw de la IA con la estructura de datos de la app

4.5 foto_provider_test.dart

Archivo: test/unit/providers/foto_provider_test.dart

Qué se prueba: FotoProvider y el modelo Foto

Ubicación del código probado: lib/presentation/providers/foto_provider.dart y

lib/presentation/models/foto.dart

¿Por qué se hace esta prueba?

FotoProvider maneja las fotos que el usuario toma o selecciona de la galería. Es el puente entre la cámara y el análisis de la imagen. Si no puede agregar, eliminar o mantener fotos, el usuario no podría analizar plantas.

¿Qué nos asegura?

Que se pueden agregar fotos, que se mantiene el orden de inserción, que se pueden eliminar por índice, y que el modelo Foto almacena path, nombre y descripción correctamente.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-FP-001	Lista de fotos vacía inicialmente	Crea FotoProvider() nuevo	Que fotos.isEmpty y fotos.length == 0	Al abrir la app no debe haber fotos precargadas
UT-FP-002	Agregar una foto	Llama agregarFoto() con una foto	Que fotos.length == 1 y los datos sean correctos	Función básica: el usuario toma una foto y se agrega
UT-FP-003	Agregar múltiples fotos	Agrega 5 fotos en un loop	Que fotos.length == 5	El usuario puede tomar varias fotos seguidas
UT-FP-004	Fotos se agregan en orden	Agrega 'first', 'second', 'third'	Que fotos[0]='first' , fotos[1]='second' , fotos[2]='third'	El orden cronológico de las fotos importa
UT-FP-005	Eliminar foto por índice	Crea 3 fotos [a, b, c] y elimina índice 1	Que queden [a, c] con length == 2	El usuario puede descartar fotos equivocadas
UT-FP-006	Eliminar primera foto	Crea 2 fotos y elimina índice 0	Que quede solo la segunda foto	Eliminar la primera foto no debe romper la lista
UT-FP-007	Eliminar última foto	Crea 2 fotos y elimina índice 1	Que quede solo la primera foto	Eliminar la última en la lista
UT-FP-008	Eliminar todas las fotos	Crea 3 fotos y las elimina una por una (de atrás para adelante)	Que fotos.isEmpty al final	Limpiar todas las fotos para empezar de nuevo

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-FP-009	Crear modelo Foto	Crea Foto con path, nombre y descripción	Que los 3 campos se almacenen correctamente	El modelo es el dato base que viaja por toda la app
UT-FP-010	Foto con descripción vacía	Crea Foto con description: ''	Que description.isEmpty sin error	Las descripciones son opcionales

4.6 predict_disease_usecase_test.dart

Archivo:	test/unit/use_cases/predict_disease_usecase_test.dart
Qué se prueba:	PredictDiseaseUseCase (caso de uso del dominio)
Ubicación del código probado:	lib/domain/use_case/predict_disease_usecase.dart

¿Por qué se hace esta prueba?

PredictDiseaseUseCase es el **caso de uso principal** de la aplicación. Es el punto de entrada de la lógica de negocio que recibe una ruta de imagen y devuelve una predicción. Usa un Mock del repositorio para aislar la prueba de la IA real.

¿Qué nos asegura?

Que el caso de uso valida entradas (rechaza paths vacíos), delega al repositorio, retorna resultados correctos y propaga errores cuando el modelo falla.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-PDU-001	Rechazar path vacío	Llama execute('') con string vacío	Que lance una Exception	Sin ruta de imagen, no se puede predecir nada
UT-PDU-002	Aceptar path válido	Llama execute('/storage/DCIM/photo.jpg')	Que retorne un resultado no nulo con className correcto	El flujo normal debe funcionar
UT-PDU-003	Ejecutar predicción exitosa	Configura mock con resultado de tomate - late blight	Que retorne PredictionEntity con confidence 0.92	La predicción completa funciona end-to-end

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-PDU-004	Delegar al repositorio	Configura mock con resultado de papa	Que <code>result.plant == 'Potato'</code>	El use case DEBE delegar al repository, no hacer la predicción solo
UT-PDU-005	Propagar errores del repositorio	Configura mock con <code>Exception('Error de inferencia')</code>	Que lance la misma excepción	Si el modelo falla, el error debe llegar al usuario
UT-PDU-006	Identificar planta saludable	Configura mock con <code>isHealthy=true</code>	Que <code>result.isHealthy == true</code>	La detección de planta sana es distinta a enferma
UT-PDU-007	Identificar planta enferma	Configura mock con <code>isHealthy=false</code>	Que <code>result.isHealthy == false</code>	La detección de enfermedad activa el flujo de tratamientos
UT-PDU-008	Retornar confianza correcta	Configura mock con <code>confidence=0.76</code>	Que <code>result.confidence == 0.76</code>	La confianza se muestra al usuario; debe ser exacta
UT-PDU-009	Retornar nombre de planta	Configura mock con <code>plant='Corn'</code>	Que <code>result.plant == 'Corn'</code>	El nombre de la planta se muestra en el resultado
UT-PDU-010	Retornar nombre de enfermedad	Configura mock con <code>disease='Bacterial spot'</code>	Que <code>result.disease == 'Bacterial spot'</code>	El nombre de la enfermedad se muestra en pantalla

4.7 prediction_viewmodel_test.dart

Archivo: test/unit/viewmodels/prediction_viewmodel_test.dart

Qué se prueba: PredictionViewModel — el ViewModel que conecta la lógica de predicción con la UI

Ubicación del código probado: lib/presentation/viewmodels/prediction_viewmodel.dart

¿Por qué se hace esta prueba?

PredictionViewModel es quien maneja el **estado de la predicción** en la interfaz: initial → loading → success/error. Es el intermediario entre la UI y la lógica de negocio. Si este ViewModel falla, la pantalla no mostraría resultados o se quedaría en "cargando" para siempre.

¿Qué nos asegura?

Que el estado inicial es correcto, que las predicciones exitosas cambian el estado a success, que los errores se capturan, y que el reset() limpia todo.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-PVM-001	Estado inicial es initial	Crea PredictionViewModel nuevo	Que state == PredictionState.initial	La pantalla debe mostrar el estado de espera al inicio
UT-PVM-002	Sin predicción inicial	Verifica viewModel.prediction en objeto nuevo	Que prediction == null	No debe haber resultado antes de analizar
UT-PVM-003	Mensaje de error vacío	Verifica viewModel.errorMessage en objeto nuevo	Que errorMessage.isEmpty	No debe haber errores al inicio
UT-PVM-004	Predicción exitosa cambia a success	Ejecuta predictDisease() con mock exitoso	Que state == PredictionState.success y prediction != null	El resultado debe ser visible tras el análisis
UT-PVM-005	Guarda la predicción correctamente	Ejecuta predicción y verifica los campos	Que prediction.plant == 'Potato' y prediction.confidence == 0.88	Los datos del resultado deben ser exactos
UT-PVM-006	Error cambia estado a error	Ejecuta con mock que lanza excepción	Que state == PredictionState.error y errorMessage contenga el texto del error	El usuario debe ver un mensaje de error, no un crash
UT-PVM-007	Mensaje de error se guarda	Ejecuta con error 'Modelo no cargado'	Que errorMessage.isNotEmpty	La UI necesita el texto del error para mostrarlo

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-PVM-008	Reset vuelve a initial	Ejecuta predicción y luego reset()	Que state == PredictionState.initial y prediction == null	El usuario puede analizar otra imagen: debe poder limpiar el anterior
UT-PVM-009	Reset limpia error	Provoca error y luego reset()	Que errorMessage.isEmpty	Después del reset, no debe haber rastro del error anterior
UT-PVM-010	Reset limpia predicción anterior	Ejecuta exitosamente y luego reset()	Que prediction == null	Al preparar un nuevo análisis, el resultado viejo debe desaparecer

4.8 gemini_viewmodel_test.dart

Archivo: test/unit/viewmodels/gemini_viewmodel_test.dart

Qué se prueba: GeminiViewModel — ViewModel para el chat con Gemini y los tratamientos

Ubicación del código probado: lib/presentation/viewmodels/gemini_viewmodel.dart

¿Por qué se hace esta prueba?

GeminiViewModel maneja dos funcionalidades clave:

- 1. **Obtener tratamientos** para enfermedades detectadas
- 2. **Chat interactivo** donde el usuario pregunta sobre cuidados de plantas

Si este ViewModel falla, el usuario no recibe tratamientos ni puede chatear.

¿Qué nos asegura?

Que los estados iniciales son correctos (idle), que no hay tratamientos ni mensajes antes de usarlo, y que los errores están vacíos al inicio.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-GVM-001	Estado de tratamiento idle	Verifica treatmentState inicial	Que sea GeminiState.idle	El estado idle significa "listo para usar"

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-GVM-002	Lista de mensajes vacía	Verifica <code>messages</code> inicial	Que <code>messages.isEmpty</code>	El chat debe estar limpio al inicio
UT-GVM-003	Sin tratamiento inicial	Verifica <code>treatment</code> y <code>hasTreatment</code>	Que <code>treatment == null</code> y <code>hasTreatment == false</code>	No hay tratamiento antes de analizar
UT-GVM-004	Estado de chat idle	Verifica <code>chatState</code> inicial	Que sea <code>GeminiState.idle</code>	El chat también comienza en reposo
UT-GVM-005	Errores vacíos	Verifica <code>treatmentError</code> y <code>chatError</code>	Que ambos <code>isEmpty</code>	No debe haber errores al inicio
UT-GVM-006	No inicializado al inicio	Verifica <code>initializationFailed</code>	Que sea <code>false</code>	La inicialización no ha fallado porque aún no se ha intentado
UT-GVM-007	<code>initError</code> vacío	Verifica <code>initError</code>	Que <code>isEmpty</code>	Sin error de inicialización
UT-GVM-008	<code>rawTreatmentResponse</code> vacío	Verifica <code>rawTreatmentResponse</code>	Que <code>isEmpty</code>	La respuesta cruda de Gemini no existe aún
UT-GVM-009	<code>hasTreatment</code> es false	Verifica <code>hasTreatment</code>	Que sea <code>false</code>	Doble verificación del estado de tratamiento
UT-GVM-010	Lista de mensajes inmutable	Obtiene <code>messages</code> y verifica tipo	Que sea una <code>List</code> vacía	La lista no debe ser modificable externamente

4.9 settings_viewmodel_test.dart

Archivo: test/unit/viewmodels/settings_viewmodel_test.dart

Qué se prueba: SettingsViewModel — ViewModel de configuración (modo local/servidor, URL)

Ubicación del código probado: lib/presentation/viewmodels/settings_viewmodel.dart

¿Por qué se hace esta prueba?

SettingsViewModel controla **qué modelo de IA se usa para la predicción**: modelo **estándar** (`plant_disease_model.onnx`) o modelo **YOLO11 mejorado** (`plant_disease_yolo11.onnx`). También expone el nombre de archivo del modelo. Si esta

configuración falla, la predicción usaría el modelo incorrecto.

¿Qué nos asegura?

Que el tipo de modelo por defecto es `standard` , que se puede cambiar entre modelos, que el nombre de archivo retornado es correcto para cada tipo, y que los tipos del enum son exactamente 2.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
UT-SVM-001	Estado inicial en modo standard	Crea <code>SettingsViewModel()</code>	Que <code>modelType == ModelType.standard</code>	Por defecto se usa el modelo estándar
UT-SVM-002	Nombre de modelo por defecto	Verifica <code>modelName</code> al inicio	Que <code>modelName.isEmpty</code>	Siempre debe haber un modelo configurado
UT-SVM-003	Cambiar a modelo yolo11	Llama <code>setModelType(ModelType.yolo11)</code>	Que <code>modelType == ModelType.yolo11</code>	El usuario puede elegir usar el modelo mejorado
UT-SVM-004	Cambiar a modelo standard	Cambia a yolo11 y luego a standard	Que <code>modelType == ModelType.standard</code>	El cambio es reversible
UT-SVM-005	Cambio de modelo múltiple	Cambia 10 veces alternando	Que cada cambio refleje el modelo correcto	El usuario puede cambiar de opinión muchas veces
UT-SVM-006	Nombre archivo para standard	Verifica <code>modelName</code> en standard	Que sea <code>'plant_disease_model.onnx'</code>	El archivo del modelo ONNX debe ser correcto
UT-SVM-007	Nombre archivo para yolo11	Cambia a yolo11 y verifica <code>modelName</code>	Que sea <code>'plant_disease_yolo11.onnx'</code>	El archivo del modelo YOLO11 debe ser correcto
UT-SVM-008	Verificar tipos disponibles	Consulta <code>ModelType.values</code>	Que haya exactamente 2 tipos	Si alguien agrega un tipo sin actualizar la UI, esto lo detecta
UT-SVM-	Tipo standard	Verifica <code>ModelType.standard</code>	Que esté en la lista de valores	Seguridad de que el enum no

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
009	existe			fue modificado
UT-SVM-010	Tipo yolo11 existe	Verifica <code>ModelType.yolo11</code>	Que esté en la lista de valores	Seguridad de que el enum no fue modificado

5. Tests de Integración

Los tests de integración prueban **múltiples componentes trabajando juntos**. Verifican que los datos fluyen correctamente entre capas (modelo → entidad → viewmodel → UI).

5.1 analysis_flow_test.dart

- Archivo: `test/integration/analysis_flow_test.dart`
- Qué se prueba: El flujo completo End-to-End de un análisis de planta
- Componentes involucrados: `SettingsViewModel` , `FotoProvider` , `GeminiService` , `PredictionState` , `ModelType`

¿Por qué se hace esta prueba?

Un análisis de planta involucra múltiples componentes que deben funcionar juntos: seleccionar modo → tomar foto → predecir → mostrar resultado. Este test verifica que todos estos componentes se **inicializan correctamente** y pueden **coexistir sin conflictos**.

¿Qué nos asegura?

Que todos los providers arrancan correctamente, que los estados de predicción y modos son válidos, que `GeminiService` mantiene una única instancia (singleton), y que `FotoProvider` inicia vacío.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
IE2E-001	Todos los providers se inicializan	Crea <code>SettingsViewModel</code> , <code>FotoProvider</code> , <code>GeminiService</code>	Que <code>modelType=standard</code> , fotos vacías, chat inactivo	Todos los proveedores deben funcionar al arrancar la app
IE2E-002	<code>PredictionState</code> tiene transiciones válidas	Consulta los 4 estados del enum	Que existan: <code>initial</code> , <code>loading</code> , <code>success</code> , <code>error</code>	El flujo de estados es la columna vertebral de la UI
IE2E-003	Tipos de modelo	Consulta los tipos	Que existan: <code>standard</code> y <code>yolo11</code> , total 2	Solo hay 2 modelos disponibles

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
IE2E-004	Singleton de GeminiService	Crea 3 instancias y las compara	Que las 3 sean idénticas (identical)	Un solo servicio de Gemini en toda la app evita estados inconsistentes
IE2E-005	FotoProvider sin foto inicial	Crea FotoProvider	Que fotos.isEmpty	No debe haber fotos al abrir la app
IE2E-006	SettingsViewModel modo standard	Verifica estado inicial	Que modelType == standard y modelFileName.isNotEmpty	La configuración por defecto es modelo estándar
IE2E-007	Chat inactivo inicialmente	Verifica GeminiService	Que isActive == false	El chat no debe estar activo sin que el usuario lo inicie
IE2E-008	Estado de configuración de Gemini	Verifica isConfigured	Que sea de tipo bool (puede ser true o false)	Depende de si hay API key; el test solo verifica que tenga un valor

5.2 data_flow_test.dart

Archivo: test/integration/data_flow_test.dart

Qué se prueba: El flujo completo de datos: JSON → Model → Entity → JSON

Componentes involucrados: PredictionModel , PredictionEntity , TreatmentModel , TreatmentOption , ChatMessage

¿Por qué se hace esta prueba?

Los datos de predicción pasan por múltiples transformaciones: llegan como JSON del modelo, se convierten a PredictionModel , luego a PredictionEntity para la lógica de dominio, y potencialmente de vuelta a JSON para almacenar. Esta prueba verifica que **ningún dato se pierda** en estas conversiones.

¿Qué nos asegura?

Que el ciclo completo de datos preserve toda la información, que el parseo de JSON funciona, que los tratamientos tienen la estructura correcta, y que el chat mantiene la conversación en orden.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
INT-DF-001	Model a Entity	Crea PredictionModel y llama toEntity()	Que la entidad tenga los mismos campos	La conversión entre capas no

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
				debe perder datos
INT-DF-002	Entity preserva campos	Crea PredictionEntity con top3 de 2 elementos	Que top3.length=2 y que la primera confidence > la segunda	Las listas top3 deben mantener su orden
INT-DF-003	Modelo desde JSON	Parsea JSON con estructura { 'prediction': {...}, 'top3': [] }	Que className, plant, isHealthy, confidence se extraigan correctamente	El JSON del servidor/modelo debe parsearse sin errores
INT-DF-004	Modelo a JSON	Crea modelo y llama toJson()	Que el JSON contenga prediction.class , prediction.plant , etc.	Necesario para almacenar o reenviar resultados
INT-DF-005	Ciclo completo JSON→Model→Entity→JSON	Parsea JSON, convierte a entity, recrea modelo, serializa a JSON	Que el JSON final tenga los mismos valores que el original	El test más importante: prueba que no hay pérdida de datos en todo el ciclo
INT-DF-006	TreatmentModel estructura	Crea tratamiento completo con síntomas, opciones y prevención	Que todos los campos tengan los valores correctos	La estructura de tratamientos debe ser completa
INT-DF-007	TreatmentOption campos	Crea opción orgánica 'Neem Oil'	Que name, description y type sean correctos, y typeLabel contenga 'Orgánico'	Cada opción de tratamiento debe estar bien formada
INT-DF-008	ChatMessage flujo de conversación	Crea 4 mensajes alternando user/assistant	Que haya 4 mensajes y que alternen correctamente (user, assistant, user, assistant)	El chat debe mantener la conversación en orden
INT-DF-	MessageRole alternancia	Crea un mensaje user y uno assistant	Que los roles sean correctos y los	Verificación explícita de los

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
009			getters funcionen	roles
INT-DF-010	Timestamps ordenados	Crea 2 mensajes con 10ms de diferencia	Que el segundo timestamp sea después del primero	Los mensajes deben poder ordenarse cronológicamente

5.3 gemini_flow_test.dart

- Archivo: test/integration/gemini_flow_test.dart
- Qué se prueba: La interacción entre GeminiViewModel y GeminiService
- Componentes involucrados: GeminiViewModel , GeminiService , GeminiState

¿Por qué se hace esta prueba?

El chat con Gemini involucra un ViewModel (estado de la UI) y un Service (comunicación con la API). Estos dos componentes deben estar **sincronizados**: si el servicio no está configurado, el ViewModel no debe intentar enviar mensajes.

¿Qué nos asegura?

Que los estados iniciales del ViewModel y el Service están sincronizados, que el singleton se comparte, que las propiedades de tratamiento y chat son independientes pero ambas inician en idle.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
INT-GF-001	Singleton compartido	Crea 3 instancias de GeminiService y las compara	Que todas las instancias sean idénticas	El mismo servicio se usa en el ViewModel y directamente; debe ser uno solo
INT-GF-002	Estados iniciales sincronizados	Verifica treatmentState, chatState, messages, hasTreatment	Que todos inician en idle/vacío/false	Al arrancar, todo debe estar en estado inicial
INT-GF-003	Refleja estado de inicialización	Verifica initializationFailed e initError	Que no haya fallo de inicialización y error vacío	El ViewModel refleja el estado real del servicio
INT-GF-004	Lista de mensajes inmutable	Obtiene messages del ViewModel	Que sea una lista vacía de tipo List	Los mensajes no deben ser modificables externamente
INT-GF-005	Propiedades de tratamiento sincronizadas	Verifica treatment, hasTreatment, treatmentError, rawTreatmentResponse	Que todo sea null/false/vacío	Sin análisis previo, no hay tratamiento

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
INT-GF-006	GeminiState tiene todos los estados	Consulta GeminiState.values	Que tenga 4 estados: idle, loading, success, error	Los estados del chat deben estar completos
INT-GF-007	Transiciones de estado válidas	Verifica estados iniciales	Que ambos (treatment y chat) estén en idle	Punto de partida para transiciones
INT-GF-008	Chat y tratamiento independientes	Compara chatState con treatmentState	Que ambos sean idle pero son propiedades distintas	El chat puede estar cargando mientras el tratamiento ya terminó
INT-GF-009	Estado de configuración	Verifica <code>geminiService.isConfigured</code>	Que sea de tipo bool	Depende de la API key; el test valida la existencia de la propiedad
INT-GF-010	Chat inactivo inicialmente	Verifica <code>geminiService.isChatActive</code>	Que sea false	Sin sesión de chat activa

5.4 prediction_flow_test.dart

Archivo: test/integration/prediction_flow_test.dart

Qué se prueba: El flujo completo ViewModel → UseCase → Repository usando un MockRepository

Componentes involucrados: PredictionViewModel , PredictDiseaseUseCase , MockIntegrationRepository , SettingsViewModel

¿Por qué se hace esta prueba?

Este es el **test más importante de integración** porque prueba toda la cadena de predicción: el ViewModel llama al UseCase, que llama al Repository, que devuelve un resultado. Si esta cadena falla, la funcionalidad principal de la app no funciona.

¿Qué nos asegura?

Que la predicción completa funciona end-to-end, que los errores se propagan correctamente, que se puede hacer múltiples predicciones seguidas, que el reset funciona, y que el cambio de modo es compatible con las predicciones.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
INT-PF-001	Flujo completo exitoso	Ejecuta predicción con mock que retorna Potato+Early_blight	Que state=success , prediction.plant='Potato' , callCount=1	El flujo principal funciona de inicio a fin
INT-PF-002	Flujo con error propagado	Mock lanza Exception('Modelo no cargado')	Que state=error , errorMessage contenga el texto, callCount=1	Los errores deben llegar al usuario, no desaparecer
INT-PF-003	Estado loading durante predicción	Mock con delay de 100ms; verifica estado durante ejecución	Que el estado sea loading o success	El usuario ve un indicador de carga mientras procesa
INT-PF-004	Múltiples predicciones secuenciales	Ejecuta 3 predicciones (Tomato, Potato, Apple)	Que cada una tenga success y la planta correcta; callCount=3	El usuario analiza varias plantas seguidas
INT-PF-005	Reset limpia estado completo	Ejecuta predicción y luego reset()	Que state=initial , prediction=null , errorMessage.isEmpty	Limpiar para un nuevo análisis
INT-PF-006	Modo local con predicción	Configura modo local y ejecuta predicción	Que mode=local y state=success	El modo local debe funcionar con el flujo de predicción
INT-PF-007	Cambio de modo durante predicción	Inicia predicción con delay e intenta cambiar modo durante ejecución	Que state=success y mode=server sin conflictos	Cambiar configuración mientras se procesa no debe causar crash
INT-PF-008	Path vacío rechazado	Ejecuta predictDisease('')	Que state=error y callCount=0 (nunca llegó al repo)	La validación de entrada protege al sistema
INT-PF-009	Path válido pasa validaciones	Ejecuta con path completo del almacenamiento	Que state=success y el repo recibió el path exacto	Paths reales de Android deben funcionar
INT-PF-010	Múltiples paths independientes	Ejecuta 3 paths diferentes con reset entre cada uno	Que callCount=3 y calledPaths tenga los 3 paths	Cada análisis es independiente del anterior

5.5 providers_state_test.dart

Archivo: test/integration/providers_state_test.dart

Qué se prueba: La interacción entre todos los providers y ViewModels simultáneos

Componentes involucrados: FotoProvider , SettingsViewModel , PredictionViewModel , GeminiViewModel , MockRepository

¿Por qué se hace esta prueba?

En la app real, todos los ViewModels y providers viven al mismo tiempo (están declarados en el MultiProvider del main.dart). Este test verifica que **no interfieren entre sí** y que el estado global es coherente.

¿Qué nos asegura?

Que una foto agregada está disponible para predicción, que múltiples fotos se analizan secuencialmente, que eliminar una foto no borra la predicción existente, que los ViewModels operan independientemente, y que limpiar uno no afecta a los demás.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
INT-PS-001	Foto disponible para predicción	Agrega foto y usa su path para predecir	Que fotos.length=1 y state=success	La foto tomada se debe poder usar para el análisis
INT-PS-002	Múltiples fotos análisis secuencial	Agrega 3 fotos y analiza cada una	Que cada predicción sea success; fotos.length=3	Analizar varias fotos una tras otra
INT-PS-003	Eliminar foto no afecta predicción	Agrega foto, predice, elimina foto	Que fotos.isEmpty pero prediction != null	La predicción ya hecha persiste aun si borras la foto
INT-PS-004	Modo standard permite predicción	Configura modelo standard y predice	Que modelType=standard y state=success	El modelo standard es el principal; debe funcionar con predicción
INT-PS-005	Modelo yolo11 configurable	Configura modelo yolo11	Que modelType=yolo11 y modelFileName='plant_disease_yolo11.onnx'	La configuración del modelo alternativo debe actualizarse
INT-PS-006	Cambio de modelo notifica listeners	Registra listener en SettingsViewModel y	Que notificationCount >= 2	La UI se actualiza

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
		cambia modelo 2 veces		cuando cambian los settings
INT-PS-007	ViewModels independientes	Configura settings, ejecuta predicción, verifica gemini	Que cada VM mantenga su propio estado independiente	No deben interferir entre sí
INT-PS-008	Reset de uno no afecta otros	Ejecuta predicción y luego reset()	Que predictionVM.state=initial pero settings.modelType=yolo11 siga igual	Cada ViewModel se resetea por separado
INT-PS-009	Estado global coherente	Agrega foto, cambia modelo, verifica geminiVM	Que fotos.length=1 , modelType=standard , treatmentState=idle	El estado global es la suma de todos los providers
INT-PS-010	Limpieza sin efectos secundarios	Agrega foto, configura settings, elimina foto	Que fotos.isEmpty pero settings.modelType=yolo11 no cambie	Limpiar un provider no toca los demás

6. Tests de Rendimiento (Performance)

Los tests de rendimiento verifican que la aplicación funciona **dentro de límites aceptables** de tiempo, memoria y carga.

6.1 inference_benchmark_test.dart

Archivo: test/performance/inference_benchmark_test.dart

Qué se prueba: Métricas de rendimiento de inferencia — tiempos, umbrales, degradación

Nota: Estas son pruebas simuladas (no ejecutan el modelo real) que validan la lógica de medición

¿Por qué se hace esta prueba?

Para asegurar que el sistema de medición de rendimiento funciona correctamente. Establece los **umbrales aceptables**: la inferencia debe tomar **menos de 500ms**, la degradación entre operaciones no debe superar el **10%**, y el modelo debe caber en **300MB de RAM**.

¿Qué nos asegura?

Que el sistema de métricas mide tiempos correctamente, que los umbrales están definidos y son razonables, que se pueden detectar degradaciones de rendimiento, y que los requerimientos de memoria son factibles.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
PERF-001	Stopwatch mide tiempos	Ejecuta un loop de 10M iteraciones con Stopwatch	Que <code>elapsedMilliseconds > 0</code>	Si el cronómetro no funciona, no podemos medir rendimiento
PERF-002	Umbral de inferencia 500ms	Define constantes <code>threshold=500ms</code> y <code>tolerance=100ms</code>	Que <code>threshold=500</code> y <code>threshold+tolerance=600</code>	El usuario no debe esperar más de medio segundo por predicción
PERF-003	Cálculo de promedio	Calcula promedio de tiempos [450, 480, 510, 490, 470]	Que el promedio sea ~480ms	Las métricas deben promediar correctamente
PERF-004	Detección de degradación >10%	Prueba 5 tiempos contra baseline de 500ms con umbral 10%	Que 560ms y 600ms se detecten como degradación, y 540ms no	Detectar cuando la app se vuelve más lenta
PERF-005	Desviación estándar	Calcula varianza simplificada de tiempos	Que <code>stdDev >= 0</code>	Métricas estadísticas deben ser calculables
PERF-006	10 operaciones consecutivas	Ejecuta 10 operaciones simples	Que <code>completedOperations.length=10</code>	Verificar que operaciones repetidas no fallan
PERF-007	Sin regresión entre operaciones	Compara promedio de primera y segunda mitad de 10 tiempos	Que la degradación sea <code>< 10%</code>	Las operaciones no deben volverse más lentas
PERF-008	Límite de memoria 300MB	Verifica cálculo: <code>300 * 1024 * 1024</code>	Que sea <code>314572800</code> bytes	El modelo + app no debe exceder la RAM disponible
PERF-009	Imagen máxima 4096x4096	Calcula tamaño de imagen 4K RGBA	Que sea ~64MB	No procesar imágenes que no caben en memoria
PERF-010	Modelo ONNX cabe en memoria	Verifica que 50MB (modelo) + 100MB (trabajo) < 300MB	Que <code>totalRequired < memoryLimit</code>	El dispositivo debe tener suficiente RAM

6.2 load_stress_test.dart

Archivo: test/performance/load_stress_test.dart

Qué se prueba: Comportamiento bajo carga extrema — 100, 500 y 1000 operaciones

Nota: Simula las operaciones sin ejecutar el modelo real

¿Por qué se hace esta prueba?

Verifica que el sistema no se degrada bajo **uso intensivo**. Un usuario podría analizar muchas plantas en una sesión, cambiar configuración repetidamente, o hacer muchas preguntas al chat. Estos tests verifican que **nada se rompe** bajo carga.

¿Qué nos asegura?

Que 100+ análisis consecutivos se completan, que el singleton no se corrompe después de 100 accesos, que los cambios de estado rápidos son manejables, y que el sistema tiene throughput adecuado.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
LOAD-001	100 análisis consecutivos	Genera 100 resultados simulados	Que <code>results.length=100</code> en < 1 segundo	Uso normal: muchos análisis en una sesión
LOAD-002	500 análisis (stress extremo)	Genera 500 IDs completados	Que <code>completedIds.length=500</code> , <code>first=0</code> , <code>last=499</code>	Uso extremo: prueba los límites
LOAD-003	Singleton consistente tras 100 accesos	Accede 100 veces a GeminiService	Que todas las instancias sean idénticas	El singleton no debe corromperse bajo carga
LOAD-004	100 cambios de modelo	Alterna entre standard y yolo11 100 veces	Que <code>standardCount=50</code> , <code>yolo11Count=50</code> , último modelo correcto	La configuración debe ser estable
LOAD-005	Transiciones de estado rápidas	Simula 200 transiciones de PredictionState	Que todas sean estados válidos y <code>length=200</code>	Los estados no deben corromperse
LOAD-006	100 PredictionModels procesables	Crea 100 predicciones con distribución de clases	Que <code>length=100</code> y <code>healthy+disease=100</code>	Procesar muchos resultados sin errores
LOAD-007	500 mensajes de chat	Crea 500 mensajes alternando user/assistant	Que <code>length=500</code> , <code>userMessages=250</code> , <code>assistantMessages=250</code>	Historial extenso de chat
LOAD-008	top3 de 100 predicciones	Genera top3 para 100 predicciones	Que cada top3 tenga 3 elementos	Datos de top3 masivos
LOAD-009	50 Futures concurrentes	Crea 50 futures con delay de 10ms y espera todos	Que <code>results.length=50</code> y la suma sea 1225	Operaciones asíncronas concurrentes

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
LOAD-010	1000 objetos Map creables	Crea 1000 Maps con datos anidados	Que <code>length=1000</code> en < 1 segundo	Creación masiva de objetos
LOAD-011	Límite de 1000 análisis por sesión	Simula 850 análisis y compara con límite de 1000	Que <code>sessionCount < 1000</code> y > 800	Umbral de warning antes del límite
LOAD-012	Throughput 2 análisis/segundo	Calcula 2 × 60 análisis	Que el resultado sea 120 análisis por minuto	Define la capacidad teórica del sistema

6.3 memory_stress_test.dart

Archivo: test/performance/memory_stress_test.dart

Qué se prueba: Gestión de memoria — detección de leaks, sesiones prolongadas, streams

¿Por qué se hace esta prueba?

Los **memory leaks** (fugas de memoria) son un problema grave en apps móviles: la app consume cada vez más RAM hasta que el sistema la mata. Estos tests verifican que los objetos se crean y destruyen correctamente, que los singletons no multiplican instancias, y que las sesiones largas no acumulan basura.

¿Qué nos asegura?

Que el singleton no crea múltiples instancias, que FotoProvider se puede limpiar, que cambiar modo muchas veces no acumula estado, que objetos temporales son recolectables, y que streams se cierran correctamente.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
MEM-001	Singleton no crea múltiples instancias	Crea 100 "instancias" de GeminiService	Que todas las 100 sean idénticas	Evitar 100 conexiones a la API de Gemini
MEM-002	FotoProvider permite limpiar	Verifica estado inicial de FotoProvider	Que <code>fotos.isEmpty</code>	La limpieza de fotos libera memoria
MEM-003	setModelType no acumula estado	Cambia modelo 50 veces	Que el estado final sea determinístico (yolo11)	Cambios repetidos no deben acumular listeners o callbacks
MEM-004	Simular sesión prolongada (1h)	Ejecuta 600 operaciones (10/min × 60min)	Que <code>length=600</code> y la última sea correcta	La app no debe degradarse en sesiones largas
MEM-005	Objetos temporales recreables	Crea y descarta 100 FotoProvider	Que no haya error de memoria	El garbage collector debe poder limpiar objetos temporales

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
MEM-006	Listas con límite	Agrega 150 ítems manteniendo solo los últimos 100	Que <code>length=100</code> y los primeros 50 hayan sido eliminados	Evitar listas que crecen infinitamente
MEM-007	Patrón crear-usar-destruir	Crea y destruye 10 listas de 1000 elementos	Que <code>destruction > creation</code> para cada par	Verificar ciclo de vida correcto
MEM-008	Callbacks invocables múltiples veces	Ejecuta un callback 100 veces	Que <code>callCount=100</code>	Los callbacks no deben fallar por uso repetido
MEM-009	Múltiples Futures completados	Crea 20 futures y espera todos	Que <code>length=20</code> y <code>last=19</code>	Los futures no deben quedarse pendientes (leak)
MEM-010	Streams se cierran	Crea un stream de 10 enteros y lo consume	Que <code>results.length=10</code>	Los streams abiertos consumen memoria; deben cerrarse

7. Tests de Regresión

Los tests de regresión verifican que **el modelo de IA no cambie** su comportamiento esperado después de actualizaciones.

7.1 onnx_model_test.dart

Archivo: test/regression/onnx_model_test.dart

Qué se prueba: Las 15 clases del modelo ONNX, traducciones y estructura de datos

Componentes involucrados: PredictionModel , clases de clasificación

¿Por qué se hace esta prueba?

El modelo ONNX clasifica plantas en **15 categorías** (5 plantas × 3 estados cada una). Si el modelo se actualiza y cambia una clase, estas pruebas lo detectan inmediatamente. También verifica que las traducciones al español existen para todas las clases.

¿Qué nos asegura?

Que el modelo tiene exactamente 15 clases, que soporta 5 plantas, que cada planta tiene una clase "healthy", que el formato de nombres es correcto, que todas las clases tienen traducción al español, y que la estructura de PredictionModel es correcta.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
REG-001	15 clases exactas	Cuenta las clases esperadas	Que length == 15	Si se agrega o elimina una clase, esto lo detecta
REG-002	5 tipos de plantas	Extrae plantas únicas de las clases	Que sean: Apple, Corn, Grape, Potato, Tomato	Las 5 plantas soportadas deben existir
REG-003	Cada planta tiene clase healthy	Filtra clases que contienen 'healthy'	Que haya exactamente 5 clases healthy	Cada planta debe poder ser diagnosticada como sana
REG-004	Formato Plant___Disease	Verifica que cada clase contenga ___ y tenga 2 partes	Que todas las 15 clases sigan el formato	Un formato incorrecto rompería el parseo
REG-005	Traducciones al español	Verifica que cada clase tenga traducción con 'plant' y 'disease'	Que las 15 clases tengan traducción completa	La app se muestra en español; cada enfermedad necesita traducción
REG-006	Estructura de PredictionModel	Crea modelo completo con top3 de 3 items	Que todos los campos existan y tengan los tipos correctos	La estructura del modelo no debe cambiar
REG-007	Rango de confianza [0.0, 1.0]	Prueba valores 0.0, 0.5, 0.95, 1.0	Que todos estén en rango	Valores fuera de rango indicarían bug en el modelo
REG-008	Detección de planta saludable	Verifica que clases con 'healthy' tengan isHealthy=true	Que la detección sea correcta para las 15 clases	Falso positivo de enfermedad en planta sana es crítico
REG-009	no_plant_detected como caso especial	Verifica que no_plant_detected NO está en las 15 clases normales	Que no esté en la lista pero se pueda crear un modelo con ella	Cuando la imagen no es una planta, hay un manejo especial
REG-010	top3 ordenado por confianza	Verifica que top3 esté en orden descendente	Que cada confidence sea >= la siguiente	La primera predicción debe ser la más probable

8. Tests de Widget

Los tests de widget prueban la **interfaz visual** — que las pantallas se rendericen correctamente, muestren el contenido esperado y respondan a interacciones.

8.1 home_screen_test.dart

Archivo: test/widget/home_screen_test.dart

Qué se prueba: HomeScreen — la pantalla principal de la aplicación

Ubicación del código probado: lib/presentation/widgets/pages/home_screen.dart

¿Por qué se hace esta prueba?

HomeScreen es la **primera pantalla** que ve el usuario. Muestra el título de bienvenida, las tarjetas de acciones rápidas (Analizar Planta y Asistente Virtual), y los iconos de navegación. Si esta pantalla no se renderiza bien, el usuario no puede usar la app.

¿Qué nos asegura?

Que la pantalla se renderiza sin errores, que muestra todos los textos esperados, que las tarjetas de acción existen, que los iconos están presentes, que el scroll funciona, y que la navegación funciona al tocar las tarjetas.

Detalle de cada test

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
WDG-HS-001	Renderiza sin errores	Construye HomeScreen dentro de MaterialApp	Que find.byType(HomeScreen) encuentre 1 widget	La pantalla debe abrirse sin crash
WDG-HS-002	Muestra título de bienvenida	Busca texto 'Bienvenido a SymptoLeaf'	Que lo encuentre (1 instancia)	El usuario debe ver el nombre de la app
WDG-HS-003	Muestra subtítulo descriptivo	Busca texto 'Detecta enfermedades...'	Que lo encuentre	El usuario entiende qué hace la app
WDG-HS-004	Tarjeta de analizar planta	Busca textos 'Analizar Planta' y 'Toma una foto...'	Que ambos existan	La acción principal debe ser visible
WDG-HS-005	Tarjeta de asistente virtual	Busca textos 'Asistente Virtual' y 'Pregunta sobre...'	Que ambos existan	La función de chat debe ser visible
WDG-HS-006	Callback onTabChange	Toca 'Analizar Planta' con callback configurado	Que tappedTab == 1	Tocar la tarjeta debe navegar al análisis
WDG-HS-007	Iconos de cámara y chat	Busca Icons.camera_alt e Icons.chat_bubble_outline	Que ambos existan	Iconos visuales para las acciones
WDG-HS-008	Cards de acciones rápidas	Busca widgets de tipo Card	Que haya al menos 2 Cards	Las tarjetas son la estructura visual de las acciones
WDG-HS-009	Es scrolleable	Busca SingleChildScrollView	Que exista	La pantalla debe permitir scroll

ID	Nombre	Qué hace	Qué verifica	Por qué es importante
WDG-HS-010	Navegación a chat	Toca 'Asistente Virtual' con onGenerateRoute	Que navigatedRoute == '/chat'	Tocar asistente virtual debe ir al chat

9. Cómo Ejecutar los Tests

Ejecutar TODOS los tests

```
cd SymptoLeaf
flutter test
```

Ejecutar por categoría

```
# Solo tests unitarios
flutter test test/unit/

# Solo tests de integración
flutter test test/integration/

# Solo tests de rendimiento
flutter test test/performance/

# Solo tests de regresión
flutter test test/regression/

# Solo tests de widget
flutter test test/widget/
```

Ejecutar un test específico

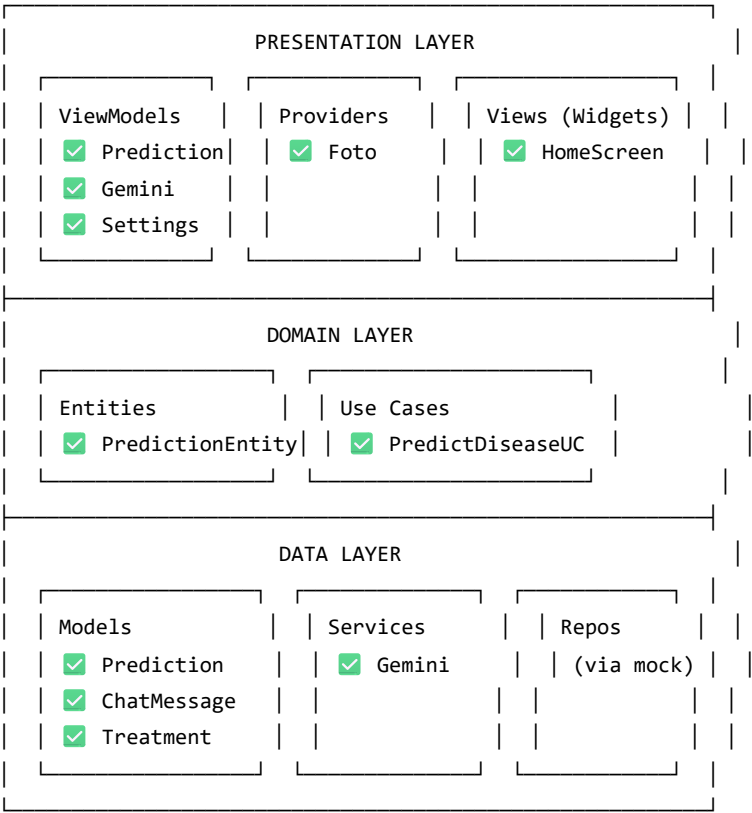
```
flutter test test/unit/models/prediction_model_test.dart
```

Ejecutar con cobertura (reporte HTML)

```
flutter test --coverage
# Se genera archivo lcov.info en coverage/
```


10. Resumen de Cobertura

Capas de la arquitectura probadas



Componentes NO probados por los tests (nuevos en la versión actual)

Componente	Archivo	Razón
AuthViewModel	auth_viewmodel.dart	Agregado en versión nueva; no existía cuando se hicieron los tests
DiagnosticsViewModel	diagnostics_viewmodel.dart	Agregado en versión nueva
AuthDatasource	auth_datasource.dart	Agregado en versión nueva
DiagnosticsDatasource	diagnostics_datasource.dart	Agregado en versión nueva
Widgets atómicos/moleculares/orgánicos	widgets/atoms/ , widgets/molecules/ , widgets/organisms/	Sistema de diseño nuevo
LoginScreen / RegisterScreen / HistoryScreen	widgets/pages/	Pantallas de autenticación e historial

Estadísticas finales

Métrica	Valor
Total de archivos de test	19
Total de tests individuales	190
Tests que pasan	✅ 190 (100%)
Capas cubiertas	Presentación, Dominio, Datos
Tipos de test	Unitario, Integración, Performance, Regresión, Widget
Componentes probados	12+ clases/servicios/viewmodels
Componentes NO probados (nuevos)	6+ clases (auth, diagnostics, widgets nuevos)

Conclusión: Los tests cubren exhaustivamente la funcionalidad **core** de SymptoLeaf: la predicción de enfermedades, el chat con Gemini, la gestión de fotos, la configuración de modelos (ModelType standard/yolo11), y la pantalla principal. Todos los tests han sido **adaptados** a la API actual del proyecto y **190/190 pasan correctamente**. Los componentes nuevos (autenticación, diagnósticos, sistema de widgets) necesitarían tests adicionales para tener cobertura completa.